Complexitylib:
An index of proved statements

19 October 2026

This tour collects 50 entries spanning the library's significant results, from machine complexity and descriptive logic to cryptography, circuit lower bounds, proof complexity, communication, coding and circuit synthesis. Some entries group closely related theorems. It is a statement index, not an exhaustive API reference or a translation of proof bodies. Each entry identifies its hypotheses, model, quantitative guarantee and scope, with links pinned to library revision `811ad02a`. Selection and wording remain open to Samuel's editorial direction.

Unless specified otherwise, encoded decision problems use an injective encoding into finite bit strings, and input size is encoded length. Class statements and reductions refer to the library's machine definitions. Circuit-family results are nonuniform unless uniformity is explicitly asserted. A circuit-size bound does not by itself certify a constructor's runtime; a decoder's correctness does not by itself certify its running time. Assumptions such as one-way functions or circuit hardness are premises of implications, not established existence results.

Source citations credit the mathematics; linked modules record formalization authors and port provenance. This index was prepared and independently reviewed with AI assistance. Lean checks the formal declarations; the prose-to-Lean comparison is a separate review, not a machine-checked equivalence proof.

Contents

1 Completeness and approximation **2**
2 Logic, diagonalization and the polynomial hierarchy **3**
3 Space, alternation and interaction **4**
4 Randomness and counting **5**
5 Cryptographic constructions **6**
6 Circuit depth, lower bounds and barriers **7**
7 Proof complexity **10**
8 Communication and formulas **10**
9 Boolean analysis and expander walks **11**
10 Reed–Solomon decoding **11**
11 Arithmetic circuits and simultaneous computation **12**
12 Circuits for iterated functions **12**

1 Completeness and approximation

1.1 Cook–Levin and its complement

SAT is NP-complete and UNSAT is coNP-complete under tidy polynomial-time many-one reductions, using the library's encoding `encCNF` of CNF formulas. Reduction efficiency is proved, not assumed.[5, Section 2.3]

`Cslib.Complexity.CookLevin.SAT_npComplete`
`Cslib.Complexity.CookLevin.UNSAT_coNPComplete`

1.2 Graph and number problems are NP-complete

Three-colorability, undirected Hamiltonian cycle, binary subset sum, and the budget decision version of traveling salesman are NP-complete under certified polynomial-time many-one reductions. Subset sum selects entries of a list of natural numbers, allowing repeated values; its binary encoding does not give strong NP-hardness. TSP uses binary natural distances and budget, and does not require symmetry or the triangle inequality.[20, 34, 54]

`Cslib.Complexity.ThreeColoring_npComplete`
`Cslib.Complexity.HamCycle_npComplete`
`Cslib.Complexity.SubsetSum_npComplete`
`Cslib.Complexity.TSP_npComplete`

Circuit evaluation. Given a one-output De Morgan circuit and an input of the matching arity, deciding whether its output is true is in P. Instances are well formed by their type; `encCircuitValue` encodes the circuit and input bits. The standard P-completeness theorem under logspace reductions is not yet formalized here. [39]

`Cslib.Complexity.CircuitValue.mem_P`

1.3 The PCP theorem

$$\mathsf{NP} = \mathsf{PCP}(O(\log n), O(1)).$$

The verifier is polynomial-time and nonadaptive, has perfect completeness and soundness at most $1/2$, and uses logarithmically many random bits and constantly many queries. In the Lean equality the bound functions are $\lfloor \log_2 n \rfloor + 1$ (with Lean's natural-number logarithm) and 1; the class definition allows constant multiples of both. It does not assert one query.[17]

`Cslib.Complexity.NP_eq_PCP_log`

1.4 A constant gap for MAX-3SAT

It is NP-hard, under polynomial-time promise reductions, to distinguish a satisfiable formula with clauses of at most three literals from one in which every assignment satisfies at most the fraction

$$1 - \frac{1}{3 \cdot 2^{77}}$$

of its clauses. Clauses are counted with multiplicity. This is the explicit gap obtained from the formalized PCP construction; no optimal approximation threshold is asserted.[6, 17]

`Cslib.Complexity.GapThreeSAT_npHard`

2 Logic, diagonalization and the polynomial hierarchy

2.1 Fagin's theorem

For a finite relational vocabulary, a class of finite structures is definable by an existential second-order sentence exactly when it is closed under isomorphism and its encoded query language belongs to NP. The encoding gives universe size in unary and framed relation truth tables; malformed tables are rejected. Both directions connect the logical semantics to the machine-backed verifier class; structures of cardinality zero and one are included.[18, 29]

`Cslib.Logic.ESO.definable_iff_isoClosed_and_queryLanguage_mem_NP`

First-order projection infrastructure. With separate numeric and input vocabularies, each projection formula selects a constant or one signed input-relation bit using numeric tests alone. For a fixed numeric structure and variable assignment, the same selected literal works for every interpretation of the input relations. The finite numeric cases are mutually exclusive and exhaustive. Projections with finite coordinate types compose, with the composite structure isomorphic to successive interpretation; quantifier-free numeric definitions and guards remain quantifier-free under composition. A concrete client is loop-free graph complement. [30, Definitions 3.3–3.6]

This is a vocabulary-parametric, relativized framework: output domains may be restricted by numeric formulas, and interpreted vocabularies are relational. It does not yet specialize to positive-dimensional full tuple universes with canonical order and BIT, certify a logspace transducer, or prove Circuit Value completeness. The classical quantifier-free convention of the cited source excludes BIT.

`Cslib.Logic.FO.ProjectionFormula.exists_literal`
`Cslib.Logic.FO.Projection.compEquiv`
`Cslib.Logic.FO.Projection.IsQF.comp`

2.2 Polynomial time is strictly smaller than exponential time

For languages of finite bit strings,

$$\mathsf{P} \subsetneq \mathsf{EXP}.$$

The separating language is given by a machine diagonalization. Here EXP is the union of deterministic time classes with bounds 2^{n^k}. The supporting time hierarchy uses a polynomial simulation gap; this entry does not claim the sharp $f \log f$ hierarchy.[5, Chapter 3]

`Cslib.Complexity.P_ssubset_EXP`

2.3 Ladner's theorem

If $\mathsf{P} \neq \mathsf{NP}$, there exists a language of bit strings in NP which is neither in P nor NP-complete under polynomial-time many-one reductions. The witness is a padded satisfiability language with a polynomial-time padding exponent. This is a conditional intermediate-degree theorem; it does not assume an intermediate problem as additional input.[5, Theorem 3.3]

`Cslib.Complexity.Ladner.exists_NP_not_P_not_NPComplete`

2.4 A Baker–Gill–Solovay oracle separation

There exists an oracle language A such that

$$\mathsf{P}^A \subsetneq \mathsf{NP}^A.$$

The classes use the library's certified polynomial-time oracle programs and oracle verifiers. This is the separating-oracle half of the relativization phenomenon, rather than a claim here of both a separating and a collapsing oracle.[7]

`Cslib.Complexity.exists_PWithOracle_ssubset_NPWithOracle`

2.5 Karp–Lipton collapse

If SAT has polynomial-size Boolean circuits on its encoded inputs, then

$$\mathsf{PH} = \Sigma_2^p.$$

In particular this follows if every NP problem, under its encoding, is the preimage of a language in P/poly. The polynomial-time computation of the self-reduction and evaluation of circuit advice are part of the development. The circuit hypothesis is nonuniform.[35]

`Cslib.Complexity.PH_eq_SigmaP_two_of_SAT`
`Cslib.Complexity.PH_eq_SigmaP_two_of_NP_subset_PPoly`

3 Space, alternation and interaction

Containment map. The proved machine-class relationships include

$$\mathsf{NL} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PH} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}, \qquad \mathsf{AP} = \mathsf{NPSPACE} = \mathsf{PSPACE}.$$

The equalities and the NL inclusion receive entries below. The links here supply the remaining arrows, including PH containment in polynomial space and the deterministic exponential-time simulation of polynomial space. Inclusion arrows do not assert strictness; the separate $\mathsf{P} \subsetneq \mathsf{EXP}$ theorem appears in the preceding section. [5, 54]

`Cslib.Complexity.P_subset_NP`
`Cslib.Complexity.NP_subset_PH`
`Cslib.Complexity.PH_subset_PSPACE`
`Cslib.Complexity.PSPACE_subset_EXP`

3.1 Nondeterministic logspace lies in polynomial time

$$\mathsf{NL} \subseteq \mathsf{P}.$$

NL is defined by native nondeterministic machines with logarithmic space bounded on every computation prefix and acceptance by an accepting path. A polynomial-time decider exists even though the nondeterministic machine's other branches need not halt. This entry does not assert the still-outstanding native $\mathsf{NL} = \mathsf{coNL}$ theorem.[54, Section 8.5]

`Cslib.Complexity.NL_subset_P`

3.2 Savitch's theorem for polynomial space

$$\mathsf{NPSPACE} = \mathsf{PSPACE}.$$

Both classes are machine-defined. Nondeterministic acceptance is by an accepting path, with every path respecting the space bound; the deterministic decider is total. This is the equality of polynomial-space classes, not a statement of the fully parameterized $\mathsf{NSPACE}(s) \subseteq \mathsf{DSPACE}(s^2)$ theorem. [54, Corollary 8.6]

`Cslib.Complexity.NPSPACE_eq_PSPACE`

3.3 TQBF is PSPACE-complete

The true closed quantified Boolean formulas are PSPACE-complete under polynomial-time many-one reductions. Membership uses an explicit-stack evaluator; hardness includes a certified polynomial-time construction of the quantified reachability formula. Formulas with free variables are excluded from TQBF. [54, Theorem 8.9]

`Cslib.Complexity.TQBF_pspaceComplete`

3.4 Alternating polynomial time equals polynomial space

$$\mathsf{AP} = \mathsf{PSPACE}.$$

AP uses the library's native alternating machines and a polynomial time bound. The equality is with the same machine-defined PSPACE used above; it is not merely an equality of abstract game predicates.[14]

`Cslib.Complexity.AP_eq_PSPACE`

3.5 Interactive proofs lie in polynomial space

$$\mathsf{IP} \subseteq \mathsf{PSPACE}.$$

IP uses a private-coin verifier realized by one fair-coin oracle machine and a polynomial clock valid against every stateful prover. Provers may be randomized and computationally unbounded; completeness is at least $2/3$ and soundness at most $1/3$. The converse $\mathsf{PSPACE} \subseteq \mathsf{IP}$ is not yet formalized here. [5, Chapter 8]

`Cslib.Complexity.IP_subset_PSPACE`

3.6 Graph non-isomorphism has a private-coin interactive proof

Graph non-isomorphism belongs to IP. The strict polynomial-time fair-coin verifier has perfect completeness and soundness at most $(17/32)^2 < 1/3$ after two rounds. It samples relabelings by sorting random vertex keys; this avoids assuming exact uniform sampling of a permutation from a fixed number of fair bits. The result is for private coins and does not establish the public-coin AM protocol. [23]

`Cslib.Complexity.GraphNonIsomorphism.isInteractiveProof_verifier`
`Cslib.Complexity.GraphNonIsomorphism.mem_IP`

4 Randomness and counting

4.1 Sipser–Gács–Lautemann

$$\mathsf{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p.$$

For the library's bounded-error polynomial-time class, membership has both an existential–universal and a universal–existential characterization with polynomially bounded bit-string witnesses and a polynomial-time predicate. [40, 53]

`Cslib.Complexity.BPP_subset_SigmaP_two_inter_PiP_two`

4.2 Toda's theorem

$$\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}.$$

Here #P counts witnesses of polynomial-time verifiers. The target class uses a polynomial-time oracle program answering decision problems via bits of one such counting function. The program is fixed before the oracle is chosen and must work for every oracle language agreeing on valid encoded queries; off-code answers are arbitrary. The target class has no separate query-count parameter. The nondeterministic-machine characterization of #P remains separate unfinished work.[58]

`Cslib.Complexity.PH_subset_PWithSharpPOracle`

4.3 Adleman's theorem

$$\mathsf{BPP} \subseteq \mathsf{P/poly}.$$

Every bounded-error polynomial-time language has a nonuniform family of polynomial-size De Morgan circuits. The result also covers encoded problems by taking the preimage of a bit-string language. It does not provide a uniform algorithm to choose the successful random strings or construct the circuits. [2, 5]

`Cslib.Circuits.Boolean.PPoly.exists_of_mem_BPP`

4.4 Derandomization from average-case hardness

Suppose a language L belongs to $\mathsf{E} = \mathsf{DTIME}(2^{O(n)})$ and there is a positive integer c such that, for every sufficiently large n, every De Morgan circuit of size at most $s_n = 2^{\lfloor n/c \rfloor}$ agrees with L on at most $1/2 + 1/s_n$ of the n-bit inputs. Then $\mathsf{P} = \mathsf{BPP}$, uniformly for encoded problems. The Nisan–Wigderson generator and exhaustive seed search have certified polynomial time. Worst-case exponential circuit hardness alone is not the hypothesis proved sufficient here.[44]
`Cslib.Complexity.P_eq_BPP_of_isAverageCaseHard`

5 Cryptographic constructions

The one-way functions and pseudorandom objects below use the library's bit-string security experiments and machine-certified efficiency predicates. Security is asymptotic against uniform strict polynomial-time probabilistic adversaries. In the one-wayness game, x is uniform among n-bit strings; the inverter receives unary n and $f(x)$ and succeeds with any preimage, even one of a different length. The function need not preserve length.

5.1 Goldreich–Levin hard-core predicate

For every one-way function f, the inner product $\langle x, r \rangle$ over $\mathbb{F}_2$ is a hard-core predicate for the padded map $(x, r) \mapsto (f(x), r)$. The formal construction splits a seed into equal halves and retains an optional spare bit, covering every seed length. The predicate is polynomial-time computable; every uniform PPT predictor has negligible advantage over a fair guess. [21]
`Cslib.Crypto.GoldreichLevin.padded_hardCore`

5.2 Pseudorandom generators from arbitrary one-way functions

Every uniform one-way function implies a deterministic polynomial-time pseudorandom generator expanding each n-bit seed to $n + 1$ bits, secure against uniform strict PPT distinguishers. The proof follows Holenstein's pseudoentropy route to the Håstad–Impagliazzo–Levin–Luby theorem. OWF existence is the sole cryptographic assumption: no entropy estimate or nonuniform advice is supplied as an extra hypothesis. The result does not claim optimized construction exponents.[27, 28]
`Cslib.Crypto.OneWay.exists_pseudorandomGenerator`

5.3 Polynomial stretch with quantitative resource bounds

Let G be a one-bit-stretch PRG and let $k : \mathbb{N} \to \mathbb{N}$ be positive and polynomial-time computable with unary input and output. Repeatedly expanding the retained n-bit seed yields a PRG with output length $n + k(n)$, secure against the same uniform strict PPT distinguishers. In particular, one-bit stretch suffices for any strictly larger, unary-polynomial-time computable target length. [8]
The construction also has a separate machine-level resource certificate. Suppose G has time/workspace bounds t_G, s_G, the count computation has bounds t_k, s_k, and every round's seed u_i satisfies $s_G(u_i) \le Q(u)$ for initial seed u. Writing $n = |u|$ and $k = k(n)$, the certified bounds are

$$\begin{aligned} \text{time} &= O\left(t_k(n) + n + k + \sum_{i<k} (t_G(u_i) + n + 1) + 1 \right), \\ \text{workspace} &= O(s_k(n) + Q(u) + n + k + 1)\,. \end{aligned}$$

The exact one-bit output-length invariant is required. The workspace bound includes the final reversal buffer; a small streaming state alone does not give an output-length-independent bound for the full construction.
`Cslib.Crypto.PRGStretch.pseudorandomGenerator`
`Cslib.Crypto.PRGStretch.computableTidilyInTimeAndSpace_stretch`

5.4 GGM and adaptive pseudorandom functions

Every uniform pseudorandom generator gives an efficiently evaluable pseudorandom function family by polynomial stretching and the GGM tree construction. Security holds against uniform adaptive strict polynomial-time oracle adversaries, not just fixed query lists. At parameter n, keys, valid queries and answers have n bits; both games answer malformed queries with the empty word. Adversaries receive the unary parameter and an empty auxiliary word. Consequently every uniform one-way function implies such a PRF. These are conditional security implications, not assertions that one-way functions exist.[22, 27]

`Cslib.Crypto.PseudorandomGenerator.exists_pseudorandomFunction`
`Cslib.Crypto.OneWay.exists_pseudorandomFunction`

6 Circuit depth, lower bounds and barriers

Circuit classes in this section are nonuniform. Unless a different cost is specified, native De Morgan circuits charge constants, NOT, AND and OR gates, with wiring and fanout free. Formula leaf count and formula depth are separate measures; unbounded fan-in bases are identified in their entries.

6.1 Shannon–Lupanov bounds and the MCSP window

Count all NOT, AND, OR and constant gates. For every $\varepsilon > 0$, all sufficiently large n, and every n-input truth table, MCSP answers yes at any integer threshold $s \geq (1+\varepsilon)2^n/n$. For every sufficiently large n, some truth table needs more than $\lfloor 2^n/n \rfloor$ gates. A finite counting bound makes the scarcity of small circuits explicit: for $n \geq 16$, at most $2^{\lfloor 2^n/4 \rfloor}$ of the 2^{2^n} truth tables have a circuit of size at most $\lfloor 2^n/(5n) \rfloor$. These are synthesis and counting statements; they do not establish the computational hardness of deciding MCSP.[41, 52]

`Cslib.MCSP.eventually_mem`
`Cslib.MCSP.eventually_exists_not_mem`
`Cslib.MCSP.ncard_setOf_mem_le`

6.2 Barrington's theorem

Nonuniform NC^1 is exactly the class of languages recognized by polynomial-length permutation branching programs of width five. An equivalent formulation uses programs over the alternating group A_5. This is a class equality with the library's nonuniform circuit families; it does not assert a uniform-family characterization.[9, 10]

`Cslib.Circuits.Boolean.NC_one_eq_polyLength_perm_five`
`Cslib.Circuits.Boolean.NC_one_eq_polyLength_alternatingGroup`

6.3 Spira's formula balancing theorem

Every De Morgan formula with $L \geq 1$ input leaves has an equivalent formula with at most $4\lceil \log_2 L \rceil$ AND/OR gates on each input-to-output path. Constants contribute zero leaves; when $L = 0$, the formal bound is zero. Negations have zero cost in this depth measure. The theorem certifies the depth of the balanced formula, not a machine runtime for constructing it.[56]

`Cslib.Circuits.Boolean.exists_formula_depthWith_le_four_mul_clog`

6.4 Khrapchenko's bound and exact parity formula size

Let a De Morgan formula be true on A and false on B, and let $E(A,B)$ be the pairs in $A \times B$ at Hamming distance one. Its input-leaf count L (constants contribute zero) satisfies

$$|E(A,B)|^2 \leq |A|\,|B|\,L.$$

In particular, parity on n bits requires at least n^2 leaves; for $n = 2^k$ this is attained. These are formula leaf bounds, not bounds on circuits with shared subcomputations.[36, 37]

`Cslib.Circuits.Formula.sq_card_hammingEdges_le`
`Cslib.Circuits.Boolean.isLeast_leafSize_parity_two_pow`

6.5 Parity lower bounds by random restrictions

A depth-d circuit of unbounded fan-in AND/OR gates and negations computing parity on n bits has at least 2^s gates whenever

$$s \geq 2, \qquad s(16s+1)^d \leq n.$$

Thus the finite theorem gives an exponential lower bound at every fixed depth. Depth counts negations and size counts every gate. The exponent obtained is of order $n^{1/(d+1)}$, weaker than the sharp classical Håstad exponent $n^{1/(d-1)}$.[4, 19, 26]

`Cslib.Circuits.Boolean.Unbounded.two_pow_le_size_of_computes_parity`

6.6 Razborov–Smolensky lower bounds

For every odd prime p, parity is not in nonuniform $\mathrm{AC}^0[p]$: adding unbounded fan-in MOD_p gates to constant-depth polynomial-size AND/OR/NOT circuits does not suffice. Quantitatively, a depth-d circuit computing parity has size at least $2^{\ell+1}/5$ whenever $\ell \geq 1$ and

$$100\big(((p-1)\ell)^d + 1\big)^2 \leq n+1.$$

The prime and depth are fixed in the class separation.[47, 55]

`Cslib.Circuits.Boolean.Smolensky.two_mul_two_pow_le_size_of_computes_parity`
`Cslib.Circuits.Boolean.Smolensky.parity_notMem_ACOMod`

6.7 Razborov's monotone clique bound

Let $p, w \geq 2$, $w+1 \leq k \leq n$, $p \leq n$, and $2w^2 \leq k-1$, and put

$$B = (p-1)^w w!, \quad A = B^2 \binom{n-w-1}{k-w-1}, \quad N = (2B+B^2)(w^2)^p (k-1)^{n-p}.$$

Every monotone AND/OR circuit for k-clique on n vertices has more than s gates if $As < \binom{n}{k}$ and $2Ns < (k-1)^n$. This states the formalized finite approximation-method bound with its parameters; no unrestricted Boolean-circuit lower bound follows from it.[46]

`Cslib.Circuits.Monotone.lt_size_of_computes_clique`

6.8 The natural-proofs barrier

Assume a family of length-doubling generators of polynomial circuit size and a fixed integer $d \geq 1$ such that, for all sufficiently large seed lengths k and every m with $m^d \leq k$, no circuit of size at most 2^m distinguishes its output from uniform with advantage greater than 2^{-m}. Then no P/poly-natural property is useful against P/poly. Here natural means containing a property of density at least $2^{-O(n)}$ whose truth-table language has circuits polynomial in the table length 2^n; useful means eventually excluding the slices of every P/poly language. The security assumption is stronger than ordinary polynomial-time PRG security. [22, 48]

`Cslib.Circuits.Boolean.not_isUseful_PPoly_of_isNatural_of_generator`

6.9 Exponentially small correlation with quadratic forms

There is a nonconstructively chosen family of bilinear Boolean functions h_n with the following property. Put

$$c_* = 1 + \frac{\pi}{6 \arccos((1+2\sqrt{2})/4)} \approx 2.78125.$$

For every real $c < c_*$, there is $\gamma > 0$ such that for all sufficiently large n, every Boolean circuit of fan-in at most two, over any Boolean basis, with at most cn positive-arity gates has

$$\left| 2 \Pr_{x \in \{0,1\}^n} [C(x) = h_n(x)] - 1 \right| \leq 2^{-\gamma n}.$$

The input is uniform, constants are not charged, and the cutoff and rate are independent of the circuit. The forms are obtained by choosing submatrix-robust matrices; no efficient construction or uniform complexity-class membership is proved. The exact coefficient above is the library's frontier-method result; related correlation-bound sources are credited separately.[15, 24, 31]

`Cslib.Circuits.Frontier.exists_eventually_abs_correlation_hardForm_le_of_lt`

6.10 Weighted-threshold activity and capacity bounds

Consider real-weight threshold gates, and count as input-reading a gate with a nonzero effective coefficient on an original input; repeated wires are aggregated before testing this condition. If a circuit computes inner product modulo two on two m-bit blocks, with its output taken from a gate, let q be its number of input-reading gates and e the maximum number of those gates outputting true on one input. Then

$$2^m \leq \left(\sum_{j=0}^{e} \binom{q}{j} \right)^2.$$

The same inequality holds with q counting all gates and e their maximum simultaneous activity.[1]

`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_sq_of_circuit_hadamardCode`
`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_le_sum_choose_gateActivity_sq_of_circuit_hadamardCode`

A separate capacity bound applies to any fixed Boolean readout of a threshold program's gate values computing f on n bits. Suppose that, across every partition of the input coordinates, every rectangle contained in either fiber of f has a side with fewer than K assignments. If q gates read original inputs in the effective sense above, then

$$2^n < 4K^2 \, 4^q.$$

This statement requires the rectangle hypothesis on both fibers; it is not an unconditional lower bound for every Boolean function.[49]

`Cslib.Circuits.Boolean.WeightedThreshold.two_pow_lt_mul_four_pow_inputGates`

7 Proof complexity

7.1 Cook–Reckhow characterization

NP equals coNP for every encoded instance type if and only if UNSAT has a polynomially bounded Cook–Reckhow proof system: a polynomial-time function from bit strings onto unsatisfiable CNF formulas, with a preimage of polynomial length for every such formula. Polynomial boundedness asserts that short proofs exist; it does not give a polynomial-time procedure for finding them. [16]
`Cslib.Complexity.ProofSystem.NP_eq_coNP_iff_exists_polynomiallyBounded`

7.2 Resolution size–width tradeoff

If a CNF formula has initial clause width at most k, at most N variables, and a resolution refutation with at most s lines, it has a refutation of some integer width w satisfying

$$w \leq k + 2\sqrt{2N \ln s} + 2.$$

Thus a large minimum refutation width forces a large refutation size. The statement concerns proof certificates, not the runtime of a resolution or DPLL solver.[11]
`Cslib.Logic.PL.CNF.exists_refutableInWidth_le_of_refutableInSize`

8 Communication and formulas

8.1 Karchmer–Wigderson correspondence

For a Boolean function on $n > 0$ bits, minimum De Morgan formula depth, counting AND/OR gates and making negations free, equals deterministic communication complexity of the following relation: Alice receives a true input, Bob a false input, and they must output a coordinate on which the inputs differ. The equality uses the library's worst-case protocol depth and extended-natural complexity. [33]
`Cslib.Circuits.Boolean.formulaDepth_eq_ecomplexity`

8.2 Yao's minimax principle

For finite input spaces and a nonempty output space, the public-coin communication complexity of a relation with worst-case error at most ε equals the supremum, over input distributions, of the minimum deterministic communication cost with distributional error at most ε. Both directions are proved, including the minimax step; the result is not only the easy averaging lower bound. Costs are worst-case numbers of communicated bits.[38, 62]
`Cslib.Communication.PublicCoin.ecomplexity_eq_iSup_distributionalComplexity`

8.3 Newman's theorem

If each party's input space has cardinality at most 2^n, then for every $\delta > 0$, private-coin communication complexity at error $\varepsilon + \delta$ is at most public-coin complexity at error ε plus

$$\left\lceil \log_2 \left\lceil n/\delta^2 \right\rceil \right\rceil$$

bits. The formal expression uses natural ceiling and `Nat.clog`, including its zero-argument convention. The shared finite sample of protocols is chosen existentially, without a certified procedure to construct it.[43]
`Cslib.Communication.PrivateCoin.ecomplexity_le_add_clog`

9 Boolean analysis and expander walks

9.1 Hypercontractivity on the Boolean cube

For a real-valued function on a finite Boolean cube with uniform probability measure, $1 \le p \le 2 \le q < \infty$, and $0 \le \rho \le \sqrt{(p-1)/(q-1)}$, the noise operator satisfies

$$\|T_\rho f\|_q \le \|f\|_p.$$

This includes the $(2,4)$ inequality at $\rho = 1/\sqrt{3}$. The indexed theorem covers the range $p \le 2 \le q$ and finite real exponents; it is not a claim of all hypercontractive parameter regimes.[13, 45]

`Cslib.BitString.lpNorm_noiseOp_le_lpNorm`

9.2 Expander-walk hitting bound

Let a finite undirected regular graph be represented by a rotation map with nonempty vertex and port sets, and let λ be its nonconstant spectral operator-norm bound. Start a walk at a uniform vertex and choose each port uniformly. If each set B_i has density at most $0 \le \mu \le 1$, then

$$\Pr[v_i \in B_i \text{ for every } 0 \le i \le t] \le \mu(\mu + \lambda(1-\mu))^t.$$

The sets may vary with time. This is a finite probability bound; machine-level error reduction and extractor evaluation costs are separate obligations. [3, 32, 59]

`Cslib.RotationMap.dens_filter_forall_walkAt_mem_le`

10 Reed–Solomon decoding

10.1 Berlekamp–Welch unique decoding

Let $\alpha_1, \dots, \alpha_n$ be distinct points of a field, and let w be a received word. If $2e + k \le n$, the decoder returns a polynomial P exactly when $\deg P < k$ and its evaluation word differs from w in at most e positions. It returns failure exactly when no such polynomial exists. The formal decoder chooses a solution of the key equations noncomputably; its correctness is proved, but polynomial decoding runtime is not certified.[25, 61]

`Cslib.Code.berlekampWelch_eq_some_iff`
`Cslib.Code.berlekampWelch_eq_none_iff`

10.2 Sudan's list-decoding guarantee

For distinct field evaluation points, $k \ge 2$, and a natural number D satisfying

$$2(k-1)n < (D+1)(D+2),$$

Sudan's decoder lists exactly the polynomials of degree below k agreeing with the received word at more than D positions. The number of listed polynomials is at most $\lfloor D/(k-1) \rfloor$. Interpolation and root containment are proved; the interpolant is chosen noncomputably, and no decoding runtime or Guruswami–Sudan multiplicity improvement is asserted.[25, 57]

`Cslib.Code.mem_sudan_iff_of_lt`
`Cslib.Code.card_sudan_mul_le`

11 Arithmetic circuits and simultaneous computation

11.1 Polynomial-size circuits for the determinant

For every commutative ring K and $n \in \mathbb{N}$, the generic $n \times n$ determinant has a division-free arithmetic circuit over K of size at most

$$3n^4 + 4.$$

Consequently its polynomial family belongs to VP over K. The same circuit evaluates determinants over every commutative K-algebra, with matrix entries supplied in row-major order. This is a gate-count assertion, not a certified uniform circuit-generation bound.[12, 60]

`Cslib.Circuits.Arithmetic.exists_circuit_det`
`Cslib.Circuits.Arithmetic.exists_circuit_det_eval`
`Cslib.Circuits.Arithmetic.detFamily_mem_VP`

11.2 Sharp circuit mass production

Fix real $0 \le \gamma < 1$ and $\varepsilon > 0$. For every sufficiently large n, every Boolean function f on n bits, and every integer $1 \le k \le 2^{\gamma n}$, one circuit computes f independently on k disjoint n-bit input blocks with nonconstant-gate cost at most

$$\left(\frac{1}{1-\gamma} + \varepsilon\right)\frac{2^n}{n}.$$

The cutoff depends on γ, ε, not on f, k. Here NOT, AND and OR each cost one and constants cost zero, unlike the total-gate convention used in other entries. The construction is nonuniform and supplies no certified runtime for choosing its tables.[51]

`Cslib.Circuits.Boolean.MassProduction.realSharpMassProduction`

12 Circuits for iterated functions

12.1 Fixed powers of arbitrary finite-state maps

Given a native De Morgan circuit of size s computing a map f on n-bit states, every fixed natural exponent T and integer $1 \le L \le 2^n$ admit a circuit for f^T of size at most

$$40000\left(Ls + (n+1)^3\left(L + \left\lfloor\frac{2^n}{L}\right\rfloor + 1\right)\right).$$

The map need not be invertible. The bound is independent of T, but T is fixed when the circuit is chosen; it is not an input to this circuit. Checkpoints are chosen nonconstructively, without a certified preprocessing runtime. [50]

`Cslib.Circuits.Boolean.Iteration.fixed_power`

12.2 An input time for an arbitrary finite-state map

With f, s, n, L as in the fixed-power result, define

$$B = Ls + (n+1)^3 \left(L + \left\lfloor \frac{2^n}{L} \right\rfloor + 1 \right).$$

For every $m \in \mathbb{N}$, one circuit on $n+m$ inputs computes $f^T(x)$ for all n-bit states x and all m-bit times $0 \le T < 2^m$, using at most

$$m(40000B + 4n)$$

gates. The time is supplied least-significant bit first. This theorem allows noninvertible maps; its linear dependence on the time width differs from the additive sparse-counter bound below. Circuit existence is nonuniform, with no certified checkpoint-construction runtime.[50]

`Cslib.Circuits.Boolean.Iteration.variable_time`

12.3 Batched iteration with a shared table

Let a size-s native De Morgan circuit compute f on n bits. For integers $0 \le r \le 2^n$ and $1 \le L \le 2^n$, put

$$B_r = rLs + (n+1)^3 \left(rL + \left\lfloor \frac{2^n}{L} \right\rfloor + 1 \right).$$

For every fixed $T \in \mathbb{N}$, one circuit applies f^T to r independent states with at most $100000B_r$ gates. With a separate m-bit input time for each state, a circuit of size at most

$$m(100000B_r + 4rn)$$

computes all r answers. The table term $\lfloor 2^n/L \rfloor$ is charged once in B_r, rather than once per state. All gates, including constants, are charged. These are nonuniform size bounds, not certified preprocessing or circuit-generation times.[50]

`Cslib.Circuits.Boolean.Iteration.batched_fixed_power`
`Cslib.Circuits.Boolean.Iteration.batched_variable_time`

Sparse-counter conventions

Write $W_n = \mathbb{Z}/2^n\mathbb{Z}$ for the n-bit words, represented least-significant bit first, and let $c(x) = x + 1$. For maps on W_n, put

$$D(f, g) = \{x \in W_n : f(x) \neq g(x)\}.$$

Circuit size below counts native De Morgan gates, including constants, comparisons, selection and addition. These are nonuniform existence results: they provide no efficient algorithm for constructing the correction table. The port follows Schlesinger's iterated-functions development; the source credits Moreno and Rivera for the related block characterization of almost-commuting permutations. [42, 50]

12.4 Maps commuting with an edited counter

For any maps $f, h : W_n \to W_n$, if h is injective and $h \circ f = f \circ h$, then there is a circuit computing all n output bits of h with size at most

$$100(n+1)(|D(f, c)| + 1).$$

There is no assumption that f is a permutation or that a small circuit for f is supplied. The assertion includes $n = 0$.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_centralizer`

12.5 Fixed signed powers and coordinate transfer

For every permutation f of W_n and every integer T, a circuit computes f^T with size at most

$$100(n+1)(|D(f,c)|+1).$$

The size bound is independent of the magnitude and sign of T. The quantifiers are "for every T, there exists a circuit": T is fixed, not an additional input to a single circuit.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_power`

Coordinate transfer. Let f, g, e be permutations of W_n such that $e(g(x)) = e(x) + 1$ for every x. Suppose circuits E and E^{-1} compute e and its inverse, respectively. For every $T \in \mathbb{Z}$, there is a circuit for f^T with size at most

$$|E| + |E^{-1}| + 100(n+1)(|D(f,g)|+1).$$

Both coordinate conversions are charged. The theorem does not assert that an arbitrary cycle has cheap coordinates or complete the general iterated-function circuit-bound program.
`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_power`

An input iteration count

Keep the notation W_n, c and D introduced above. Here a circuit reads an n-bit initial state x and an m-bit nonnegative integer T, both least-significant bit first. The permutation is fixed when choosing the circuit. The chart and circuit are chosen nonuniformly; no efficient preprocessing or circuit-generation algorithm is asserted.[50]

12.6 Sparse-counter input times and cycle labels

For every permutation f of W_n and every $m \in \mathbb{N}$, there is a native De Morgan circuit with $n+m$ inputs and n outputs computing

$$(x, T) \longmapsto f^T(x), \qquad 0 \le T < 2^m,$$

of size at most

$$1000(n+3)(|D(f,c)|+m+1).$$

The dependence on the edit count and time width is additive inside the parentheses. One circuit handles all initial states and all m-bit times. The bound includes constants, comparisons and selection gates, and permits $n = 0$ or $m = 0$.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_variable_time`

Canonical coordinates. For every permutation f of W_n, there is a circuit with n inputs and $n + 2(n+2)$ outputs computing the canonical cycle representative, cycle length and phase of the input. Its size is at most

$$500(n+3)(|D(f,c)|+1).$$

The representative is the numerically least word on the cycle. The phase is the forward distance from that representative, modulo the cycle length. The period and phase each occupy $n+2$ bits, so even a period of 2^n fits.
`Cslib.Circuits.Boolean.Iteration.sparse_counter_labels`

Coordinate transfer. Let f, g, e be permutations of W_n satisfying $e(g(x)) = e(x) + 1$, with supplied circuits E and E^{-1} for e and its inverse. For every time width m, a circuit computing $(x, T) \mapsto f^T(x)$ has size at most

$$|E| + |E^{-1}| + 1000(n + 3)(|D(f, g)| + m + 1).$$

The m time bits pass through unchanged to the iteration stage. Both state coordinate conversions are charged once, independently of m.
`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_variable_time`

References

[1] Threshold circuits as decision trees and finite-state programs. Research draft, second pass, 2026. URL https://github.com/SamuelSchlesinger/threshold-circuits/blob/791fa2749317fbcc7b5c5b030a1e6fadd28cc44c/threshold_circuit_representations.tex. October 18, 2026; repository revision 791fa2749317fbcc7b5c5b030a1e6fadd28cc44c.

[2] Leonard Adleman. Two theorems on random polynomial time. In *19th Annual Symposium on Foundations of Computer Science*, pages 75–83. IEEE Computer Society, 1978. doi: 10.1109/SFCS.1978.37.

[3] M. Ajtai, J. Komlós, and E. Szemerédi. Deterministic simulation in LOGSPACE. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, pages 132–140. ACM, 1987. doi: 10.1145/28395.28410. URL https://doi.org/10.1145/28395.28410.

[4] Miklós Ajtai. Σ^1_1-formulae on finite structures. *Annals of Pure and Applied Logic*, 24(1):1–48, 1983. doi: 10.1016/0168-0072(83)90038-6.

[5] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.

[6] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998. doi: 10.1145/278298.278306.

[7] Theodore Baker, John Gill, and Robert Solovay. Relativizations of the $\mathcal{P} =? \mathcal{NP}$ question. *SIAM Journal on Computing*, 4(4):431–442, 1975. doi: 10.1137/0204037.

[8] Boaz Barak. CS 127: Pseudorandomness, 2016. URL https://www.boazbarak.org/cs127spring16/chap03-pseudorandom-generators. Length extension for pseudorandom generators.

[9] David A. Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1. *Journal of Computer and System Sciences*, 38(1):150–164, 1989.

[10] David A. Mix Barrington and Denis Thérien. Finite monoids and the fine structure of NC^1. *Journal of the ACM*, 35(4):941–952, 1988. doi: 10.1145/48014.63138.

[11] Eli Ben-Sasson and Avi Wigderson. Short proofs are narrow—resolution made simple. *Journal of the ACM*, 48(2):149–169, 2001. doi: 10.1145/375827.375835. URL https://doi.org/10.1145/375827.375835.

[12] Richard S. Bird. A simple division-free algorithm for computing determinants. *Information Processing Letters*, 111(21–22):1072–1074, 2011. doi: 10.1016/j.ipl.2011.08.006.

[13] Aline Bonami. Étude des coefficients de Fourier des fonctions de $L^p(G)$. *Annales de l'Institut Fourier*, 20(2):335–402, 1970. doi: 10.5802/aif.357.

[14] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114–133, 1981. doi: 10.1145/322234.322243.

[15] Ruiwen Chen and Valentine Kabanets. Correlation bounds and #sat algorithms for small linear-size circuits. In *Computing and Combinatorics (COCOON 2015)*, Lecture Notes in Computer Science, pages 211–222. Springer, 2015. doi: 10.1007/978-3-319-21398-9_17.

[16] Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *The Journal of Symbolic Logic*, 44(1):36–50, 1979. doi: 10.2307/2273702.

[17] Irit Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3):12, 2007. doi: 10.1145/1236457.1236459.

[18] Ronald Fagin. Generalized first-order spectra and polynomial-time recognizable sets. In Richard M. Karp, editor, *Complexity of Computation*, volume 7 of *SIAM-AMS Proceedings*, pages 43–73. 1974.

[19] Merrick Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. *Mathematical Systems Theory*, 17(1):13–27, 1984. doi: 10.1007/BF01744431.

[20] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.

[21] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, pages 25–32, 1989. doi: 10.1145/73007.73010. URL https://www.wisdom.weizmann.ac.il/~oded/X/gl.pdf.

[22] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986. URL https://www.wisdom.weizmann.ac.il/~oded/X/ggm-jacm.pdf.

[23] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):690–728, 1991. doi: 10.1145/116825.116852.

[24] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Circuit size lower bounds and #sat upper bounds through a general framework. In *41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016)*, volume 58 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 45:1–45:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. doi: 10.4230/LIPIcs.MFCS.2016.45.

[25] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. Essential coding theory, 2019. URL https://cse.buffalo.edu/faculty/atri/courses/coding-theory/book/. Draft book, 2019.

[26] Johan Håstad. Almost optimal lower bounds for small depth circuits. In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing*, pages 6–20, 1986. doi: 10.1145/12130.12132.

[27] Thomas Holenstein. Pseudorandom generators from one-way functions: A simple construction for any hardness. In *Theory of Cryptography (TCC 2006)*, 2006. URL https://crypto.ethz.ch/publications/files/Holens06.pdf.

[28] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999. doi: 10.1137/S0097539793244708. URL https://doi.org/10.1137/S0097539793244708.

[29] Neil Immerman. *Descriptive Complexity*. Graduate Texts in Computer Science. Springer, 1999. doi: 10.1007/978-1-4612-0539-5.

[30] Neil Immerman and Susan Landau. The complexity of iterated multiplication. *Information and Computation*, 116(1):103–116, 1995. URL https://people.cs.umass.edu/~immerman/pub/mult.pdf.

[31] Stasys Jukna. *Extremal Combinatorics: With Applications in Computer Science*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2 edition, 2011. doi: 10.1007/978-3-642-17364-6.

[32] Nabil Kahale. Eigenvalues and expansion of regular graphs. *Journal of the ACM*, 42(5): 1091–1106, 1995. doi: 10.1145/210118.210136. URL https://doi.org/10.1145/210118.210136.

[33] Mauricio Karchmer and Avi Wigderson. Monotone circuits for connectivity require super-logarithmic depth. *SIAM Journal on Discrete Mathematics*, 3(2):255–265, 1990.

[34] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[35] Richard M. Karp and Richard J. Lipton. Some connections between nonuniform and uniform complexity classes. In *Proceedings of the 12th Annual ACM Symposium on Theory of Computing*, STOC 1980, pages 302–309, 1980. doi: 10.1145/800141.804678.

[36] V. M. Khrapchenko. Complexity of the realization of a linear function in the class of π-circuits. *Mathematical Notes of the Academy of Sciences of the USSR*, 9:21–23, 1971. Russian original: Matematicheskie Zametki 9(1), 35–40.

[37] V. M. Khrapchenko. Method of determining lower bounds for the complexity of π-schemes. *Mathematical Notes of the Academy of Sciences of the USSR*, 10:474–479, 1971. doi: 10.1007/BF01747074. Russian original: Matematicheskie Zametki 10(1), 83–92.

[38] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[39] Richard E. Ladner. The circuit value problem is log space complete for P. *ACM SIGACT News*, 7(1):18–20, 1975. doi: 10.1145/990518.990519.

[40] Clemens Lautemann. BPP and the polynomial hierarchy. *Information Processing Letters*, 17 (4):215–217, 1983. doi: 10.1016/0020-0190(83)90044-3.

[41] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya Vysshikh Uchebnykh Zavedenii. Radiofizika*, 1(1):120–140, 1958. URL https://radiophysics.unn.ru/sites/default/files/papers/1958_1_120.pdf. In Russian.

[42] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5:1949, 2016. doi: 10.1186/s40064-016-3638-7. URL https://doi.org/10.1186/s40064-016-3638-7. Article 1949.

[43] Ilan Newman. Private vs. common random bits in communication complexity. *Information Processing Letters*, 39(2):67–71, 1991. doi: 10.1016/0020-0190(91)90157-D.

[44] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994. doi: 10.1016/S0022-0000(05)80043-1.

[45] Ryan O'Donnell. *Analysis of Boolean Functions*. Cambridge University Press, 2014. doi: 10.1017/CBO9781139814782.

[46] Alexander A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Soviet Mathematics Doklady*, 31:354–357, 1985. Russian original: Doklady Akademii Nauk SSSR 281(4), 798–801.

[47] Alexander A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41 (4):333–338, 1987. doi: 10.1007/BF01137685.

[48] Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55(1):24–35, 1997. doi: 10.1006/jcss.1997.1494.

[49] Samuel Schlesinger. Algebraic circuit formalizations in Complexitylib, 2026. URL https://github.com/SamuelSchlesinger/complexitylib/tree/2ad6cb481b70ee6e65e078dffb76b9856be199d3/Complexitylib/Algebraic. Source of ported Lean proofs, revision 2ad6cb481b70ee6e65e078dffb76b9856be199d3.

[50] Samuel Schlesinger. Square-root circuit bounds for iterated functions, 2026. URL https://github.com/SamuelSchlesinger/iterating-functions/tree/c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b/lean. Source of ported Lean proofs, revision c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b.

[51] Samuel Schlesinger. Boolean circuit mass production formalization, 2026. URL https://github.com/SamuelSchlesinger/algebraic-circuits/tree/3998dc48823f7d2798aa814f2cbfbf3ceba146fa/Algebraic. Source of ported Lean proofs, revision 3998dc48823f7d2798aa814f2cbfbf3ceba146fa.

[52] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi: 10.1002/j.1538-7305.1949.tb03624.x. URL https://deeplearning.cs.cmu.edu/F24/document/readings/Shannon49.pdf.

[53] Michael Sipser. A complexity theoretic approach to randomness. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 330–335, 1983. doi: 10.1145/800061.808762. URL https://doi.org/10.1145/800061.808762.

[54] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.

[55] Roman Smolensky. Algebraic methods in the theory of lower bounds for Boolean circuit complexity. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, pages 77–82, 1987. doi: 10.1145/28395.28404.

[56] Philip M. Spira. On time-hardware complexity tradeoffs for Boolean functions. In *Proceedings of the Fourth Hawaii International Symposium on System Sciences*, pages 525–527, 1971.

[57] Madhu Sudan. Decoding of Reed Solomon codes beyond the error-correction bound. *Journal of Complexity*, 13(1):180–193, 1997. doi: 10.1006/jcom.1997.0439.

[58] Seinosuke Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991. doi: 10.1137/0220053. URL https://doi.org/10.1137/0220053.

[59] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1–3):1–336, 2012. doi: 10.1561/0400000010.

[60] Leslie G. Valiant. Completeness classes in algebra. In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, STOC '79, pages 249–261. ACM, 1979. doi: 10.1145/800135.804419.

[61] Lloyd R. Welch and Elwyn R. Berlekamp. Error correction for algebraic block codes, 1986. U.S. Patent 4,633,470.

[62] Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, FOCS 1977, pages 222–227, 1977. doi: 10.1109/SFCS.1977.24.